

Relatório Técnico: Sistema de Reserva de Estacionamento Inteligente

1. Sumário

2.	Arquitetura do Sistema	2
2.1.	Visão Geral	2
2.2.	Backend (Spring Boot).....	3
2.2.1.	Camadas da Aplicação	3
2.2.2.	Componentes Transversais (Cross-Cutting Concerns).....	4
2.3.	Arquitetura do Frontend (React)	5
3.	Modelo de Dados.....	6
4.	Casos de Uso	9
4.1.	Caso de Uso: Cadastro de Novo Usuário	9
4.2.	Caso de Uso: Gerenciamento de Veículos	10
4.3.	Caso de Uso: Realização de Reserva de Vaga.....	12
4.4.	Caso de Uso: Liberação Automática de Reservas Expiradas.....	13
5.	Decisões Técnicas.....	14
5.1.	Arquitetura e Stack de Tecnologias: Separação de Responsabilidades	14
5.2.	Decisões de Backend (Spring Boot)	14
5.2.1.	Segurança: Autenticação via JWT e Autorização baseada em Roles	14
5.2.2.	Persistência de Dados: JPA e Gerenciamento de Migrations com Flyway	15
5.2.3.	Processamento Assíncrono: Tarefas Agendadas com @Scheduled	16
5.2.4.	Fluxo de Cadastro em Duas Etapas	16
5.3.	Decisões de Frontend (React)	17
5.3.1.	Tipagem Estática com TypeScript.....	17

5.3.2.	Gerenciamento de Estado e Comunicação com API	17
5.3.3.	Tratamento de CORS.....	18
6.	Limitações e Próximos Passos	18
6.1.	Limitações Atuais.....	18
6.1.1.	Falta de Histórico de Reservas	18
6.1.2.	Segurança de Acesso a Dados do Cliente.....	19
6.1.3.	Repetição de Código e Inconsistência nos Controllers	19
6.1.4.	Ausência de um Sistema de Pagamentos.....	19
6.1.5.	Comunicação Unidirecional (Cliente-Servidor).....	19
6.2.	Próximos Passos e Sugestões de Melhoria.....	20
6.2.1.	Implementar Histórico de Reservas (Soft Delete)	20
6.2.2.	Corrigir a Falha de Segurança no ClienteController.....	20
6.2.3.	Refatorar o VeiculoController	20
6.2.4.	Integrar um Gateway de Pagamento.....	20
6.2.5.	Implementar Comunicação em Tempo Real com WebSockets	21
7.	Conclusão.....	21

2. Arquitetura do Sistema

2.1. Visão Geral

O sistema é construído sobre o modelo **Cliente-Servidor**, composto por duas aplicações principais:

1. **Frontend (Cliente):** Uma **Single Page Application (SPA)** desenvolvida em **React com TypeScript**. É responsável por toda a interface do usuário, gerenciamento de estado local e comunicação com o backend.
2. **Backend (Servidor):** Uma **API RESTful** desenvolvida em **Java com Spring Boot**. É responsável pela lógica de negócio, processamento de dados, segurança e persistência.
3. **Banco de Dados:** Um banco de dados relacional **PostgreSQL**, que serve como a fonte de verdade para todos os dados da aplicação.

A comunicação entre o frontend e o backend ocorre exclusivamente via **HTTP/S**, utilizando o padrão **REST** com payloads em formato **JSON**.

2.2. Backend (Spring Boot)

O backend segue uma **arquitetura em camadas (Layered Architecture)**, um padrão clássico que promove a separação de interesses e o baixo acoplamento entre os componentes.

2.2.1. Camadas da Aplicação

- **Camada de Apresentação (Controller - web):**
 - **Responsabilidade:** Expor os endpoints da API REST, receber requisições HTTP, validar os dados de entrada (`@Valid`) e delegar a execução para a camada de serviço.
 - **Implementação:** Utiliza `RestController`s como o [UsuarioController.java](#) e [ClienteController.java](#). Esta camada não contém lógica de negócio; seu foco é o protocolo HTTP e a serialização/desserialização de dados (DTOs).
 - **Segurança:** A autorização é aplicada aqui usando anotações do Spring Security (`@PreAuthorize`), definindo quais perfis (`ROLE_ADMIN`, `ROLE_CLIENTE`) podem acessar cada endpoint.
- **Camada de Serviço (Service - service):**
 - **Responsabilidade:** Orquestrar a lógica de negócio principal da aplicação. É o coração do sistema.
 - **Implementação:** Classes anotadas com `@Service`, como `UsuarioService` e `ClienteService`. Elas interagem com a camada de repositório para manipular os dados.
 - **Transacionalidade:** A anotação `@Transactional` é usada para garantir a atomicidade das operações de negócio que envolvem múltiplas escritas no banco de dados (ex: criar uma reserva e atualizar o status da vaga).
 - **Processos**
 - Assíncronos:** O [AgendadorDeReservas.java](#) demonstra o uso de `@Scheduled` para executar tarefas em segundo plano

(background jobs), como a limpeza de reservas expiradas, desacoplando essa lógica do fluxo principal do usuário.

- **Camada de Acesso a Dados (Repository - repository):**

- **Responsabilidade:** Abstrair a comunicação com o banco de dados. Define as operações de CRUD (Create, Read, Update, Delete) e consultas customizadas.
- **Implementação:** Interfaces que estendem JpaRepository (Spring Data JPA), como ReservaRepository e RecursoRepository. O Spring Data JPA gera a implementação em tempo de execução, simplificando drasticamente o código de persistência.

- **Camada de Domínio (Model - model):**

- **Responsabilidade:** Representar as entidades de negócio do sistema.
- **Implementação:** Classes POJO (Plain Old Java Object) anotadas com @Entity (JPA), como Cliente, Usuario, Reserva. Elas mapeiam diretamente as tabelas e relacionamentos do banco de dados.

2.2.2. Componentes Transversais (Cross-Cutting Concerns)

- **Segurança (security, jwt):**

- A segurança é baseada em **Spring Security** e **JSON Web Tokens (JWT)**.
- **Autenticação:** O usuário envia credenciais (email/senha), o backend as valida e retorna um token JWT.
- **Autorização:** Para cada requisição subsequente, o frontend envia o token no cabeçalho Authorization: Bearer <token>. Um filtro do Spring Security intercepta o token, valida sua assinatura e extrai os dados do usuário (ID, perfil), que são usados para decisões de controle de acesso com @PreAuthorize.

- **Gerenciamento de Exceções (infra.exception):**

- O sistema utiliza um manipulador de exceções global (@RestControllerAdvice).

- Quando uma exceção de negócio (ex: `PhoneUniqueViolationException`) é lançada em qualquer camada, ela é capturada por este handler, que a converte em uma resposta HTTP padronizada com o status code apropriado (ex: 409 Conflict) e uma mensagem de erro clara em JSON. Isso centraliza o tratamento de erros e fornece um feedback consistente para o frontend.
- **Configuração de CORS (infra.cors):**
 - O arquivo [CorsConfig.java](#) configura a política de **Cross-Origin Resource Sharing**. Ele permite explicitamente que a aplicação frontend (rodando em `http://localhost:3000`) faça requisições para o backend, uma configuração de segurança essencial em arquiteturas distribuídas.
- **Versionamento de Banco de Dados (Flyway):**
 - Conforme o [README.md](#), o projeto utiliza **Flyway**. Os scripts de migração SQL localizados em `resources/db/migration` garantem que a estrutura do banco de dados seja criada e atualizada de forma automática e consistente em qualquer ambiente (desenvolvimento, teste, produção).

2.3. Arquitetura do Frontend (React)

O frontend é uma **Single Page Application (SPA)**, o que significa que a aplicação é carregada uma única vez, e a navegação entre as páginas é gerenciada no lado do cliente, proporcionando uma experiência de usuário mais fluida e rápida.

- **Gerenciamento de Componentes (components):**
 - A interface é construída com componentes reutilizáveis (ex: [AuthLayout.tsx](#)), promovendo consistência visual e manutenibilidade.
- **Roteamento (react-router-dom):**
 - A navegação entre as diferentes "páginas" (views) da aplicação, como `/login`, `/cadastro`, `/reserva/:id`, é controlada pelo React Router.
- **Comunicação com a API (api):**

- **Axios** é utilizado para realizar as chamadas HTTP para o backend. Uma instância centralizada (api/api.js) é configurada para interceptar requisições e adicionar automaticamente o token JWT no cabeçalho Authorization, simplificando a comunicação com endpoints protegidos.
- **Gerenciamento de Estado (context, hooks):**
 - O estado global da aplicação, como as informações do usuário autenticado e o token, é gerenciado através da **Context API** e **Hooks** (useState, useEffect). Isso permite que diferentes componentes acessem e reajam a mudanças no estado compartilhado de forma eficiente.
- **Tipagem (types, TypeScript):**
 - O uso de **TypeScript** adiciona segurança de tipos ao código, definindo interfaces (ex: [Veiculo.ts](#)) que espelham os DTOs do backend. Isso reduz erros em tempo de execução e melhora a experiência de desenvolvimento ao fornecer autocompletar e verificação estática.

A arquitetura do Smart Parking IoT é moderna, robusta e bem-estruturada. A separação clara entre um backend stateless com lógica de negócio e um frontend reativo para a interface do usuário segue as melhores práticas de desenvolvimento de software. O uso de tecnologias como Spring Security com JWT, Spring Data JPA, Flyway e uma arquitetura em camadas no backend, combinado com React, TypeScript e Axios no frontend, resulta em um sistema seguro, escalável e de fácil manutenção.

3. Modelo de Dados

Para o armazenamento de dados, foi adotado um modelo **relacional**, implementado em um banco de dados SQL. Essa abordagem foi escolhida devido à natureza estruturada dos dados e à necessidade crítica de garantir a integridade, consistência e atomicidade (propriedades ACID) das transações, especialmente nas operações de reserva de vagas.

O modelo é centrado na entidade usuarios, que se relaciona com clientes (para dados pessoais), veiculos (para os carros cadastrados) e reservas (o

registro da alocação de um recurso). A entidade recursos representa as vagas de estacionamentos. A seguir, detalhamos cada uma das tabelas que compõem o banco de dados do sistema.

Tabela: usuarios

Armazena as credenciais de acesso e informações de perfil dos usuários do sistema.

Campo	Tipo de Dado	Descrição e Restrições
id	bigint	Identificador único (Chave Primária).
username	varchar(100)	Login do usuário. Não pode ser nulo e deve ser único.
password	varchar(200)	Senha criptografada do usuário. Não pode ser nula.
role	varchar(25)	Nível de permissão (ROLE_ADMIN, ROLE_CLIENTE).
activated	boolean	Indica se a conta está ativa.
data_criacao	timestamp(6)	Data e hora da criação do registro.
data_modificacao	timestamp(6)	Data e hora da última modificação.
criado_por	varchar(255)	Usuário que criou o registro.
modificado_por	varchar(255)	Usuário que modificou o registro.

Tabela: clientes

Contém os dados pessoais dos clientes, complementando a tabela usuarios.

Campo	Tipo de Dado	Descrição e Restrições
id	bigint	Identificador único (Chave Primária).
usuario_id	bigint	Chave estrangeira para usuarios (relação 1:1).
nome	varchar(100)	Nome completo do cliente. Não pode ser

		nulo.
cpf	varchar(14)	CPF do cliente. Não pode ser nulo e deve ser único.
telefone	varchar(15)	Telefone de contato. Não pode ser nulo e deve ser único.
data_criacao	timestamp(6)	Data e hora da criação do registro.
data_modificacao	timestamp(6)	Data e hora da última modificação.
criado_por	varchar(255)	Usuário que criou o registro.
modificado_por	varchar(255)	Usuário que modificou o registro.

Tabela: veiculos

Armazena informações sobre os veículos cadastrados por cada usuário.

Campo	Tipo de Dado	Descrição e Restrições
id	bigint	Identificador único (Chave Primária).
usuario_id	bigint	Chave estrangeira para usuarios, indicando o proprietário.
placa	varchar(255)	Placa do veículo. Não pode ser nula e deve ser única.
marca	varchar(255)	Marca do veículo (ex: Fiat, Chevrolet).
modelo	varchar(255)	Modelo do veículo (ex: Uno, Onix).

Tabela: recursos

Representa as vagas de estacionamento disponíveis para reserva.

Campo	Tipo de Dado	Descrição e Restrições
id	bigint	Identificador único (Chave Primária).
nome	varchar(255)	Nome/código da vaga (ex: "A-01"). Único e não nulo.

numero_vaga	integer	Número da vaga. Único e não nulo.
tipo	varchar(255)	Tipo da vaga (CARRO_PEQUENO, MOTO, VAGA_PCD, etc.).
status	varchar(255)	Status atual da vaga (ex: "DISPONIVEL", "OCUPADA").

Tabela: reservas

Tabela transacional que registra as reservas de vagas feitas pelos usuários.

Campo	Tipo de Dado	Descrição e Restrições
id	bigint	Identificador único da reserva (Chave Primária).
usuario_id	bigint	Chave estrangeira para usuarios (quem fez a reserva).
veiculo_id	bigint	Chave estrangeira para veiculos (veículo usado na reserva).
recurso_id	bigint	Chave estrangeira para recursos (vaga que foi reservada).
horario_inicio	timestamp(6)	Início da vigência da reserva. Não pode ser nulo.
horario_fim	timestamp(6)	Fim da vigência da reserva.

to disponíveis.

4. Casos de Uso

4.1. Caso de Uso: Cadastro de Novo Usuário

O processo de cadastro é o ponto de entrada para novos clientes no sistema. O fluxo foi desenhado para ser simples e seguro, separando a criação da credencial de acesso (usuário) da criação do perfil detalhado (cliente).

- **Frontend (src/pages/Cadastro.tsx):**

- A página de cadastro apresenta um formulário solicitando apenas **email** (username) e **senha**.
 - Utiliza a biblioteca react-hook-form para validação em tempo real (e.g., campos obrigatórios, comprimento mínimo da senha).
 - Ao submeter o formulário, uma requisição POST é enviada para o endpoint /api/v1/usuarios.
 - Em caso de sucesso, o usuário é notificado via SweetAlert2 e redirecionado para a página de login, com a instrução de que deve completar seu perfil após o login.
 - O tratamento de erros captura respostas do backend (como email já existente) e as exibe de forma clara para o usuário.
- **Backend:**
 - O UsuarioController (não fornecido, mas inferido) recebe a requisição POST /api/v1/usuarios.
 - Valida os dados recebidos (e.g., formato do email, complexidade da senha).
 - Verifica se o username (email) já existe no banco de dados para evitar duplicidade.
 - A senha é codificada usando BCryptPasswordEncoder antes de ser persistida.
 - O novo usuário é salvo com o perfil (Role) padrão de CLIENTE.
 - **Observação:** Após o login, o usuário precisará acessar uma área de "Meu Perfil" para criar a entidade Cliente associada, enviando dados como CPF e nome, conforme exposto no ClienteController.

4.2. Caso de Uso: Gerenciamento de Veículos

Para que um cliente possa fazer uma reserva, ele precisa primeiro cadastrar um ou mais veículos. Esta funcionalidade é central para a agilidade do processo de reserva.

- **Frontend (src/pages/GerenciarVeiculos.tsx):**

- A página exibe um formulário para adicionar/editar veículos e uma lista dos veículos já cadastrados pelo usuário.
- Ao carregar, envia uma requisição GET para /api/v1/veiculos (através do VeiculoService) para buscar a lista de veículos do usuário autenticado.
- **Adicionar Veículo:** O formulário envia uma requisição POST para /api/v1/veiculos com placa, marca e modelo.
- **Editar Veículo:** Clicar em "Editar" preenche o formulário com os dados do veículo selecionado e, ao salvar, envia uma requisição PUT para /api/v1/veiculos/{id}.
- **Remover Veículo:** Apresenta um modal de confirmação para evitar exclusões acidentais. Se confirmado, envia uma requisição DELETE para /api/v1/veiculos/{id}.
- A interface gerencia estados de carregamento (loading) e exibe feedback (sucesso ou erro) de forma clara, melhorando a experiência do usuário.

- **Backend (VeiculoController - não fornecido):**

- Todos os endpoints são protegidos e associados ao usuário autenticado via token JWT.
- GET /veiculos: Retorna a lista de veículos associados ao id do usuário extraído do token.
- POST /veiculos: Valida os dados (e.g., formato da placa) e verifica a unicidade da placa no sistema antes de criar e associar o novo veículo ao usuário.
- PUT /veiculos/{id}: Valida a posse do veículo (se o veículo pertence ao usuário que está fazendo a requisição) antes de atualizar os dados.
- DELETE /veiculos/{id}: Valida a posse do veículo antes de removê-lo.

4.3. Caso de Uso: Realização de Reserva de Vaga

Este é o fluxo principal do sistema, permitindo que um usuário autenticado com veículos cadastrados reserve uma vaga de estacionamento disponível.

- **Frontend (src/pages/Reserva.tsx):**

- A página é acessada com o ID de um recurso (vaga), por exemplo, /reserva/12.
- useEffect dispara requisições para buscar os detalhes da vaga (GET /api/v1/recursos/{id}) e a lista de veículos do usuário (GET /api/v1/veiculos).
- O usuário seleciona um de seus veículos em um campo <select>.
- Ao clicar em "Reservar Agora", uma requisição POST é enviada para /api/v1/reservas. O corpo da requisição contém o recursoid e os dados do veículo selecionado.
- Em caso de sucesso, uma mensagem de confirmação é exibida e o usuário é redirecionado para a página "Minhas Reservas" após 3 segundos.
- O tratamento de erro é robusto, informando sobre conflitos (e.g., vaga já reservada, status 409) ou outros problemas.

- **Backend (ReservaController - não fornecido):**

- O endpoint POST /api/v1/reservas é protegido e requer autenticação.
- O serviço de reserva realiza uma série de validações críticas em uma única transação:
 1. Verifica se o recurso (vaga) existe e se seu status é available.
 2. Verifica se o usuário já não possui outra reserva ativa para evitar múltiplas reservas simultâneas.
 3. Se todas as validações passarem, o status do recurso é alterado para occupied.
 4. Uma nova entidade Reserva é criada, associando o usuário, o recurso e o horário de início.

- Se qualquer validação falhar, a transação é revertida e uma exceção de negócio (e.g., `RecursoNotAvailableException`, `UserAlreadyHasReservationException`) é lançada, resultando em uma resposta HTTP apropriada (como 409 Conflict).

4.4. Caso de Uso: Liberação Automática de Reservas Expiradas

Para garantir a disponibilidade das vagas, o sistema precisa de um mecanismo para liberar aquelas cujas reservas expiraram ou não foram utilizadas.

- **Frontend:**

- Não há interação direta do usuário com este processo. A liberação é percebida indiretamente quando uma vaga que estava ocupada volta a aparecer como available na listagem de vagas.

- **Backend**

(`src/main/java/br/edu/ifba/park/iot/backend/service/AgendadorDeReservas.java`):

- Esta é uma funcionalidade puramente de backend, implementada com o mecanismo de tarefas agendadas do Spring.
- A classe `AgendadorDeReservas` contém um método anotado com `@Scheduled(fixedRate = 60000)`, que é executado automaticamente a cada 60 segundos.
- O método `liberarReservasExpiradas()`:
 1. Consulta o `ReservaRepository` usando o método `findByHorarioFimBefore(limite)`, buscando por reservas que deveriam ter terminado há mais de 15 minutos (tolerância).
 2. Para cada reserva expirada encontrada, ele obtém o `Recurso` associado.
 3. Altera o status do `Recurso` para "available".
 4. Salva a alteração no `Recurso` e, em seguida, **deleta** a entidade `Reserva`.

5. Logs detalhados (Slf4j) registram a atividade do agendador, facilitando o monitoramento e a depuração.

5. Decisões Técnicas

5.1. Arquitetura e Stack de Tecnologias: Separação de Responsabilidades

A decisão mais fundamental foi a adoção de uma **arquitetura Cliente-Servidor desacoplada**, com duas bases de código distintas:

- **Backend (Spring Boot):** Uma API RESTful *stateless* responsável exclusivamente pela lógica de negócio, segurança e persistência de dados.
- **Frontend (React):** Uma Single Page Application (SPA) responsável por toda a interface do usuário, gerenciamento de estado no cliente e consumo da API.

Justificativa:

- **Manutenibilidade e Escalabilidade:** Permite que as equipes de frontend e backend trabalhem de forma independente. Cada parte pode ser escalada, atualizada ou até mesmo substituída sem impactar a outra, desde que o contrato da API (os endpoints REST) seja mantido.
 - **Flexibilidade:** A mesma API backend pode, no futuro, servir a outros clientes, como um aplicativo mobile (Android/iOS) ou outros sistemas, sem nenhuma alteração.
 - **Tecnologias Especializadas:** Permite o uso das melhores ferramentas para cada finalidade: Java e Spring Boot para um backend robusto e seguro, e React com TypeScript para uma interface de usuário moderna, reativa e tipada.
-

5.2. Decisões de Backend (Spring Boot)

5.2.1. Segurança: Autenticação via JWT e Autorização baseada em Roles

O sistema implementa um fluxo de segurança robusto utilizando **Spring Security** e **JSON Web Tokens (JWT)**.

- **Fluxo de Autenticação:** O usuário envia username e password. O backend os valida e, se corretos, gera um token JWT assinado que contém o ID do usuário e seus perfis (Roles). Este token é retornado ao cliente.
- **Fluxo de Autorização:** Para cada requisição a um endpoint protegido, o frontend envia o token no cabeçalho Authorization: Bearer <token>. Um filtro do Spring Security intercepta a requisição, valida a assinatura do token e carrega o contexto de segurança do usuário.
- **Controle de Acesso Fino:** A anotação @PreAuthorize é usada extensivamente nos controllers (ex: UsuarioController, ClienteController) para definir regras de acesso declarativas e legíveis.
 - @PreAuthorize("hasRole('ADMIN')"): Permite acesso apenas a administradores.
 - @PreAuthorize("hasRole('CLIENTE') AND #id == authentication.principal.id"): Permite que um cliente acesse apenas seus próprios recursos, garantindo o isolamento de dados entre usuários.

Justificativa:

- **Stateless:** O JWT torna a API *stateless*, pois o servidor não precisa armazenar informações de sessão. O estado da autenticação está contido no próprio token, o que simplifica a escalabilidade horizontal.
- **Segurança e Clareza:** O uso de @PreAuthorize centraliza as regras de segurança junto aos endpoints, tornando o código mais seguro e fácil de auditar.

5.2.2. Persistência de Dados: JPA e Gerenciamento de Migrations com Flyway

A interação com o banco de dados **PostgreSQL** é gerenciada por duas tecnologias chave.

- **Spring Data JPA:** Abstrai o acesso ao banco de dados, permitindo que os desenvolvedores trabalhem com objetos Java (@Entity) em vez de escrever SQL manualmente. O uso de interfaces JpaRepository gera automaticamente as operações de CRUD.
- **Flyway:** Conforme indicado no [README.md](#) e nas propriedades (spring.flyway.enabled=true), o Flyway é usado para gerenciar o

versionamento do schema do banco de dados. Os scripts SQL em `resources/db/migration` garantem que a estrutura do banco seja criada e atualizada de forma consistente em qualquer ambiente. A configuração `spring.jpa.hibernate.ddl-auto=validate` (inferida do [README.md](#)) complementa essa abordagem, fazendo com que o Hibernate valide se o mapeamento das entidades corresponde ao schema gerenciado pelo Flyway, prevenindo inconsistências.

Justificativa:

- **Produtividade:** O Spring Data JPA reduz drasticamente o código boilerplate de acesso a dados.
- **Confiabilidade:** O Flyway garante que a estrutura do banco de dados seja uma fonte de verdade confiável e versionada, eliminando problemas de "funciona na minha máquina" relacionados ao banco de dados.

5.2.3. Processamento Assíncrono: Tarefas Agendadas com `@Scheduled`

A classe [AgendadorDeReservas.java](#) demonstra uma decisão importante para a autossuficiência do sistema.

- **Implementação:** O método `liberarReservasExpiradas` é anotado com `@Scheduled(fixedRate = 60000)`, fazendo com que o Spring o execute automaticamente a cada 60 segundos.
- **Lógica:** Esta tarefa verifica reservas que expiraram, libera a vaga associada (mudando seu status para `available`) e remove o registro da reserva.

Justificativa:

- **Automação e Resiliência:** Automatiza uma tarefa de manutenção crítica, garantindo que as vagas não fiquem bloqueadas indefinidamente. Isso melhora a disponibilidade de recursos sem a necessidade de intervenção manual.
- **Desacoplamento:** A lógica de expiração é desacoplada do fluxo de reserva do usuário, tornando o sistema mais eficiente e responsivo.

5.2.4. Fluxo de Cadastro em Duas Etapas

Uma decisão de design de negócio refletida na arquitetura é a separação entre `Usuario` e `Cliente`.

1. **Criação do Usuário (/api/v1/usuarios):** O fluxo inicial, como visto em [Cadastro.tsx](#), cria apenas a entidade Usuario com email e senha. Este é o registro de credencial.
2. **Criação do Cliente (/api/v1/clientes):** Após o login, o usuário deve completar seu perfil, criando a entidade Cliente associada, que contém dados pessoais como nome e CPF.

Justificativa:

- **Redução de Fricção:** Simplifica o formulário de cadastro inicial, aumentando a taxa de conversão. O usuário só precisa fornecer dados adicionais quando for realmente utilizar uma funcionalidade que os exija.
 - **Separação de Contextos:** A entidade Usuario pertence ao contexto de segurança (autenticação), enquanto Cliente pertence ao contexto de negócio (reservas, pagamentos, etc.). Essa separação é uma boa prática de design.
-

5.3. Decisões de Frontend (React)

5.3.1. Tipagem Estática com TypeScript

O uso de **TypeScript** em todo o projeto frontend (.tsx, .ts) é uma decisão estratégica.

Justificativa:

- **Segurança de Tipos:** Define contratos claros para componentes, props e dados da API (ex: types/Veiculo.ts). Isso captura muitos erros em tempo de compilação, antes mesmo de o código ser executado no navegador.
- **Melhora a Experiência do Desenvolvedor:** Fornece autocompletar inteligente, refatoração segura e documentação implícita, tornando o código mais fácil de entender e manter, especialmente em equipes.

5.3.2. Gerenciamento de Estado e Comunicação com API

- **Context API + Hooks:** O estado global, como os dados do usuário autenticado, é gerenciado pela Context API (useAuth). Isso evita o "prop drilling" e fornece uma maneira limpa de compartilhar estado entre componentes.

- **Axios com Interceptadores:** Uma instância centralizada do Axios (`api/api.js`) é configurada para interceptar todas as requisições e adicionar automaticamente o token JWT ao cabeçalho `Authorization`.

Justificativa:

- **Simplicidade e Eficiência:** A Context API é nativa do React e suficiente para o gerenciamento de estado da aplicação, evitando a complexidade de bibliotecas externas como Redux para este escopo de projeto.
- **Código Limpo (DRY):** O interceptador do Axios centraliza a lógica de autenticação das chamadas de API, evitando repetição de código em cada requisição.

5.3.3. Tratamento de CORS

O arquivo [CorsConfig.java](#) no backend é a contraparte de uma decisão de arquitetura distribuída. Ele permite explicitamente que a origem do frontend (`http://localhost:3000`) acesse a API.

Justificativa:

- **Segurança do Navegador:** É uma configuração de segurança obrigatória para permitir que um domínio (frontend) faça requisições para outro (backend), conforme exigido pela Same-Origin Policy dos navegadores. A configuração é específica e não abre a API para origens desconhecidas.

6. Limitações e Próximos Passos

6.1. Limitações Atuais

Apesar de ser uma aplicação robusta e bem-estruturada, foram identificados alguns pontos que podem ser aprimorados ou que representam limitações no escopo atual.

6.1.1. Falta de Histórico de Reservas

A implementação atual do [AgendadorDeReservas.java](#) **exclui permanentemente** as reservas expiradas do banco de dados.

- **Impacto:** Impede a criação de funcionalidades importantes como "Histórico de Reservas" para o usuário, além de impossibilitar a geração

de relatórios gerenciais (ex: taxa de ocupação, vagas mais utilizadas, horários de pico) e auditorias.

- **Arquivo Relevante:** [AgendadorDeReservas.java](#)

6.1.2. Segurança de Acesso a Dados do Cliente

O ClienteController permite que um usuário com perfil CLIENTE acesse dados de *qualquer* outro cliente se souber o ID correspondente.

- **Impacto:** Representa uma falha de segurança (Insecure Direct Object Reference - IDOR), pois um cliente não deveria poder visualizar ou modificar dados de outro. A regra de autorização deveria garantir que um cliente só possa acessar seus próprios dados.
- **Arquivo Relevante:** [ClienteController.java](#)

6.1.3. Repetição de Código e Inconsistência nos Controllers

O VeiculoController repete a lógica para obter o usuário autenticado em todos os seus métodos, utilizando SecurityContextHolder. Além disso, em caso de conflito (placa duplicada), ele retorna um corpo de resposta vazio (body(null)), o que é inconsistente com o resto da API, que utiliza uma estrutura ErrorMessage padronizada.

- **Impacto:** Dificulta a manutenção e viola o princípio DRY (Don't Repeat Yourself). A falta de uma resposta de erro padronizada prejudica o tratamento de erros no frontend.
- **Arquivo Relevante:** [VeiculoController.java](#)

6.1.4. Ausência de um Sistema de Pagamentos

O fluxo de reserva está completo, mas não há integração com um sistema de pagamentos. O modelo de negócio de um estacionamento depende da cobrança pelo tempo de uso.

- **Impacto:** Limita a aplicação a um protótipo funcional, não sendo viável para um cenário de produção real.

6.1.5. Comunicação Unidirecional (Cliente-Servidor)

O sistema depende que o cliente (frontend) consulte o servidor para obter atualizações de status das vagas. Não há um mecanismo para que o servidor "empurre" atualizações em tempo real para os clientes.

- **Impacto:** Se uma vaga for liberada, os usuários só verão a atualização quando recarregarem a página ou fizerem uma nova consulta, o que não proporciona uma experiência verdadeiramente em tempo real.

6.2. Próximos Passos e Sugestões de Melhoria

Com base nas limitações identificadas, os seguintes passos são recomendados para a evolução do projeto.

6.2.1. Implementar Histórico de Reservas (Soft Delete)

Em vez de deletar a reserva, adicione um campo de status à entidade Reserva (ex: ATIVA, FINALIZADA, EXPIRADA).

- **Ação:** Modificar o `AgendadorDeReservas` para apenas atualizar o status da reserva e liberar o recurso, sem excluir o registro.
- **Benefício:** Habilita a criação de um histórico para usuários e a geração de relatórios analíticos para administradores.

6.2.2. Corrigir a Falha de Segurança no `ClienteController`

Ajuste as regras de autorização para garantir que um cliente só possa acessar seus próprios dados.

- **Ação:** Adicionar uma verificação na anotação `@PreAuthorize` que compara o ID do recurso solicitado com o ID do usuário autenticado.
- **Benefício:** Garante o isolamento de dados e a privacidade dos usuários, corrigindo uma vulnerabilidade de segurança crítica.

6.2.3. Refatorar o `VeiculoController`

Refatore o controller para eliminar código repetido e padronizar as respostas de erro.

- **Ação:** Utilize a anotação `@AuthenticationPrincipal` para injetar os detalhes do usuário diretamente nos métodos, e crie exceções customizadas para lidar com conflitos de placa, que serão capturadas pelo handler global de exceções.
- **Benefício:** Código mais limpo, legível, manutenível e consistente com o restante da aplicação.

6.2.4. Integrar um Gateway de Pagamento

Adicione um fluxo de pagamento ao finalizar uma reserva.

- **Ação:**
 1. No backend, criar endpoints para iniciar e confirmar pagamentos, integrando com um serviço como Stripe ou Mercado Pago.

2. No frontend, criar uma tela de checkout onde o usuário insere os dados de pagamento.
 3. A reserva só seria confirmada após o sucesso do pagamento.
- **Benefício:** Transforma o protótipo em um produto comercialmente viável.

6.2.5. Implementar Comunicação em Tempo Real com WebSockets

Adote WebSockets para uma experiência de usuário mais dinâmica.

- **Ação:**
 1. No backend, configurar o **Spring WebSockets** para criar um tópico (ex: /topic/vagas).
 2. Quando o status de uma vaga mudar (seja por uma reserva, liberação ou sensor IoT), o backend publicaria uma mensagem nesse tópico.
 3. No frontend, utilizar uma biblioteca como **StompJS** ou **Socket.IO** para se inscrever no tópico e atualizar a interface do usuário em tempo real, sem a necessidade de polling.
- **Benefício:** Proporciona uma experiência de usuário moderna e reativa, refletindo instantaneamente o estado real do estacionamento.

7. Conclusão

O sistema de reserva de estacionamento inteligente representa uma solução inovadora para otimizar o uso de espaços urbanos. Combinando tecnologias modernas como IoT, computação em nuvem e interfaces móveis, o sistema oferece praticidade, segurança e eficiência. A evolução contínua do projeto visa atender às necessidades dos usuários e contribuir para a mobilidade urbana sustentável.